

SwiftUI Color Matching Game – Complete Study Notes

This PDF contains all important explanation points discussed about the SwiftUI Color Matching Game project.

It explains the structure, logic, and SwiftUI concepts in a simple way for study and revision.

1. PROJECT STRUCTURE OVERVIEW

The project contains four main Swift files:

- Color_Matching_GameApp.swift – Entry point of the application
- ContentView.swift – Main user interface and interaction logic
- GameLogic.swift – Pure game logic functions
- Tile.swift – Data model representing one tile

This separation follows good software design principles:
UI is separated from logic and data.

2. Color_Matching_GameApp.swift – APPLICATION ENTRY POINT

This file is the starting point of the app.

Key Points:

- @main tells Swift that this struct is the starting point
- The struct conforms to the App protocol
- WindowGroup creates the main app window
- ContentView() is loaded as the first screen

Flow:

App Launch → Color_Matching_GameApp → WindowGroup → ContentView

This file normally does not contain game logic.
Its only job is to launch the application.

3. ContentView.swift – MAIN GAME SCREEN

This file contains:

- All user interface elements
- Game interaction handling
- State variables
- Visual grid layout
- Button actions

Important SwiftUI Concepts Used:

@State Variables

These variables automatically refresh the UI when changed.

Main State Variables:

tiles – array of all Tile objects

selectedIndexes – which tiles user tapped

clickCount – number of user clicks

successMessage – win or joker message
currentLevel – selected game level

Computed Property:

gridSize decides grid dimensions based on level:

Level 1 → 3x3

Level 2 → 5x5

Level 3 → 7x7

UI Components Used:

- VStack – vertical layout
- Text – to show title and messages
- Picker – to select level
- LazyVGrid – to display tiles in grid
- Button – restart game

Main User Interactions:

- Tap on tile
- Flip tile
- Match checking
- Level changing
- Restarting game

Important Functions:

startGame()

- Creates new tiles
- Resets counters
- Shuffles tiles

tileTapped()

- Handles tap logic
- Flips tiles
- Checks match
- Handles Joker tile

checkWin()

- Confirms if all tiles are matched

4. GameLogic.swift – GAME RULES FILE

This file contains only logic, no UI.

Functions:

generateTiles()

- Calculates number of tiles
- Selects random colors
- Creates matching pairs
- Adds Joker if needed
- Shuffles tiles
- Returns array of Tile objects

isMatch()

- Receives two selected tiles
- Compares their colors
- Returns true or false

Why static functions?

Because they are utility functions and do not require an object.

5. Tile.swift – DATA MODEL

This file defines what a single tile is.

struct Tile: Identifiable

Properties:

id – unique identifier

color – actual tile color

isFlipped – whether tile is face up

isMatched – whether tile is already matched

isJoker – special joker tile

This struct represents one square in the game.

6. HOW ALL FILES WORK TOGETHER

Tile.swift

Defines what a tile is.

GameLogic.swift

Creates and checks tiles.

ContentView.swift

Displays tiles and handles user actions.

Color_Matching_GameApp.swift

Launches the application.

Flow:

User Tap → ContentView → GameLogic → Tile Model → UI Update

7. IMPORTANT SWIFTUI CONCEPTS LEARNED

- @State for UI updates
 - Struct based UI
 - Views and Modifiers
 - LazyVGrid for grids
 - Data binding
 - Separation of concerns
 - Event handling
 - DispatchQueue for delays
 - Computed properties
-

8. SUMMARY

This project demonstrates:

- Basic SwiftUI app structure
- Game logic implementation
- State management
- Grid-based UI
- Clean project architecture

It is an excellent beginner SwiftUI learning project covering:

UI design

Logic separation

User interaction

Game development basics

END OF NOTES